IBM BOB HACKATHON · TRACK 2 — MODERNIZE WHAT MATTERS

ParityProof

Modernize COBOL to Java and **prove the new code behaves exactly like the original** — a copybook-aware runner diffs the COBOL reference and the Java twin field-by-field, on identical inputs.

✔ It runs

Real COBOL compiled under GnuCOBOL; Java 21 twin; edge-case input generator.

✔ It proves

100% field parity, and it catches real divergence (injected rounding bug → 90%, field pinpointed).

✔ It is central to IBM Bob

Bob extracts the rules, plans the migration, writes the Java — and a hook runs parity after every edit.

Team: Abdul Qaadir (Java lead · Bob driver) · Satish (Cloud · CI) · Karthik (Business analysis · story)

THE PROBLEM

Banks want to modernize. They're afraid to.

80%+ of transactions

Run on IBM i / IBM Z, much of it in COBOL batch that has run untouched for decades.

Senior COBOL developers are retiring

The pattern is a knowledge cliff — the people who understand the business rules are leaving.

Modernization risk is the blocker

"Tests pass" on generated code means nothing until you've shown the new output matches the old.

The question every bank asks first is not "did you generate code?" — it's "is it the same?"

WHY THIS WINS

Many teams work with Bob. Few prove the output is correct.

The first thing any bank wants is a guarantee the rewrite is behaviorally identical. We built the mechanism that delivers it.

Most teams

Show Bob generating Java → judge sees code, must trust it.

ParityProof

Shows Bob generating Java **and a running proof it matches the COBOL byte-for-byte**.

Judging edge

Correctness + explainability is the defensible innovation, not the generation itself.

100%
field parity — 20/20

90%
→ FAIL when a rounding slip is injected — field pinpointed

20/20
fields diffed on 5 edge-case records

WHAT WE BUILT

ParityProof pipeline

- **COBOL reference runtime** — GnuCOBOL Docker image runs the original batch on flat-file inputs.
- **Business-rule catalog** — Bob explains each program; Karthik sanity-checks the rules.
- **Java 21 modernized program** — Spring Boot CLI job; copybook becomes a fixed-width record class.
- **Edge-case input generator** — zero/negative balances, half-cent rounding, max field sizes, leap dates, empty files.
- **Parity runner** — runs both on every input, diffs every field by the copybook layout → results.json.
- **CI + dashboard** — GitHub Actions fails the build unless parity is 100%; Azure-hosted live dashboard.

Every output keeps the original fixed-width format so the diff is exact.

LIVE PROOF

Parity catches real bugs

Interest batch, acct 1: COBOL `00000043208` (432.08). Java with HALF_DOWN rounding: `00000043207` (432.07).

CORRECT
HALF_UP → 100% pass, 20/20 fields

INJECTED BUG
HALF_DOWN → 90% FAIL, INTEREST + NEW-BAL flagged

One preview: **speed of verification** — a full COBOL + Java compile-and-parity run completes in seconds in CI, so every Bob edit is checked on the spot via a hook. Usability and performance measured by the runner, not a guess.

HOW IBM BOB IS CENTRAL

Bob in the loop at every step

Ask — extract rules

"COBOL explainer" skill produces a consistent business-rule catalog with source-line citations.

Plan — migration plan

Per-program plan: copybook → record, COMP-3 → BigDecimal, truncation handling; scope locked with the team.

Code — write the Java

Bob generates the Java twin; the **hook runs the parity suite after every edit** so a mismatch fails immediately.

Governance

Rules file (COMP-3 → BigDecimal, COBOL truncation, cite source lines) + project skill + parity-hook.

Adoption evidence

Bob drives each rewrite (Ask/Plan/Code), screen-recorded; Bobalytics usage exported for the submission.

TECH STACK

Built with

GnuCOBOL 3.1 | Java 21 / BigDecimal | Spring Boot 3 | Docker (GnuCOBOL image) | GitHub Actions CI | Python (results pipeline) | Azure Static Web Apps | IBM Bob (Ask / Plan / Code) | CardDemo (source app)

Correctness infrastructure

Copybook-driven field diff; BigDecimal arithmetic; fixed-width I/O so diffs are exact.

Repeatability

Docker + CI reproduce GnuCOBOL and the parity run on any machine, any push.

Evidence machine-readable

results.json drives the live dashboard + Bobalytics numbers in one place.

FUTURE ROADMAP & LEARNINGS

Next steps

- **More CardDemo programs** — transaction posting, statements (interest is the proof slice).
- **COMP-3 / packed-decimal parity** — the genuinely hard COBOL→BigDecimal cases.
- **PCI scan** — card numbers in logs, weak crypto; Bob applies the fixes.
- **Live Azure dashboard** — parity % per program/rule with drill-down.

Key learning

Parity-as-a-test-harness changes how a migration is trusted: it turns an article of faith ("Bob's code works") into a measurable, verifiable guarantee — which is exactly the evidence a bank asks for before signing off.